Архітектурний аудит Wind

Поле	Значення
Об’єкт	Wind (`wismh/wind-engine`), CMake/namespace `engine`
Зріз	`origin/main` @ `14b1945` (merge dev → `main`, wind-282/283)
Версія збірки	`0.2.10-dev` (локальна; релізний рядок задає `CMakeLists.txt`)
Дата	25 жовтня 2026, Europe/Berlin
Тип	Архітектура й межі, не стиль і не баг-хант
Попередній зріз	вересень 2026, `c1ba89f` (wind-113)

Це оновлення аудиту під актуальний скоуп продукту. Wind більше не позиціонує себе як «маленький embeddable 2D-рушій». README і `docs/tech/architecture/Scope.md` кажуть: 2D-first production engine з базовим 3D, редактор як хост, гра як гість-модуль, SDK-релізи. Оцінки нижче — проти цієї заяви, не проти старої.

Вердикт

Архітектура витримує нову заяву. Ядро більше не виглядає випадково зібраним навколо однієї гри. Воно зібране навколо редактора й SDK: `EngineHost`, `EngineServices`, `Worlds`, `GameModule` (Play/Stop), розкол `register_*_systems`, процесний `Presentation`, `build_id` замість `kApiEpoch`.

Це вже не мінімальна бібліотека. Це batteries-included 2D-комбайн із editor-host. Для production-титулу це нормальний вибір. Ризик змістився: не «гра просочилась у рушій», а публічне API й command buffer починають нести форму редактора.

Загалом: 8 / 10. Утримувана, узагальнена архітектура — так. Маленький embeddable Wind — ні, і документи це вже не обіцяють.

Оцінки

Шкала 1–10. «Було» — перший аудит (~wind-91). «wind-113» — вересень 2026. «Зараз» — `14b1945`.

Вимір	Було	wind-113	Зараз	Коментар
Architecture	7	7.5	8	Principles/Scope/Boundaries, host, Worlds, розкол систем
Abstraction	6.5	6.5	7	`EngineServices`, `IPresentation`, HTTP/process/capture; GL усе ще RTTI
Modularity	6	6	7.5	simulation / UI / audio; editor vs engine; модульна документація
Engine / Game	7.5	8.5	8.5	Гра — гість. Inspector/profiler у публічних заголовках — тертя
Change resilience	6	6.5	7.5	`build_id`, `unload_catalog`, `detach_game`
Extensibility	6	6.5	7.5	Опційна реєстрація систем; `IPaint`; серіалізація компонентів гри
C++	7	7	7.5	`expected`, `ENGINE_API`, C ABI модуля
Ownership	6.5	6.5	7	Unload каталогу й модуля задокументовані; GPU `shared_ptr` лишився
Testability	8	8.5	9	~127 `*_test.cpp` у `tests/` + editor tests + fixtures
Performance	6.5	7	7.5	Job pool, lazy UI, власний UI renderer замість NanoVG

Що змінилось у продукті

Ідентичність

Раніше SDD казав: маленька статична бібліотека, гра через `add_subdirectory`, editor/networking/ієрархія/3D — out of scope.

Зараз:

- Гра не тримає цей репозиторій як submodule. Вона знаходить SDK (`find_package(Wind)`).
- `ENGINE_EDITOR` збирає `engine` як shared library; експорт лишається static.
- `wind_editor` вантажить `game.dll` на Play і вивантажує на Stop.
- Базовий 3D (perspective camera, quaternion, Lambert, до 4 directional lights) — in scope.
- HTTP, child processes, job system, hierarchy (`Parent` + derived `WorldTransform`), localization — in scope.
- Редактор in progress, але вже хост: проєкт, збірка, play, inspector, UI tree, profiler, export desktop.

Аудит більше не карає рушій за спрайти, партикли, 9-slice чи CSS `var()`. Це примітиви 2D-рушія. Карати варто лише те, що ламає нову межу: гра не повинна залежати від SDL/glad, інструменти редактора не повинні ставати геймплейним API.

Композиція, яку аудит просив

Кілька старих «критичних» дірок закриті свідомо, не випадково.

Закритий `register_engine_systems`. Функція лишилась як зручний бандл, але ядро розколото:

- `register_simulation_systems` — physics, animator, particles, `Phase::Transform`, world render
- `register_ui_systems` — input, dock, pointer targets, bind, UI render
- `register_audio_systems` — лише за наявності assets + audio

`Worlds::enable_ui` / `enable_audio` підключають UI/audio на конкретний світ. Редакторський світ і ігровий світ більше не зобов'язані нести весь комбайн однаково. Це пряма відповідь на головний акцент вересневого аудиту.

God-object ініціалізації. `Engine<GameT>` тонкий. `EngineHost` володіє runtime, fatal, assets, input, audio, haptics, HTTP, worlds. Редактор сидить на тому ж хості (`attach_game` / `detach_game`). `EngineServices` — явна сумка посилань у конструктор гри, не `Engine::get_audio()`.

Мертвий `IGame::on_draw`. Прибрано разом із `on_update` / `on_fixed_update`. Гра — `world()`, `on_start`, `on_quit`, вікно, іконка, опційний seed. Симуляція йде системами.

Розміри вікон / mouse consumed. Принцип 14: size, pointer, consumption живуть на процесній `Presentation`, не як окремі world-ctx сінглтони. `bind_presentation` садить вказівник у світ. Dual `WindowSize` ctx зник раніше (wind-107); зараз це доведене до процесного об'єкта.

Ресурси. `AssetsDb::unload_catalog` викидає записи одного кореня й їхній кеш. Редактор це робить на Stop. UI-шрифти/картинки як і раніше ліниві (`ensure_ui_*`). `kApiEpoch` замінено на `kBuildId` (хеш публічних заголовків + компілятор + конфіг).

Ієрархія. Не поле в `Transform` (це прямо забороняють Principles). Окремий `Parent`, derived `Children` / `WorldTransform`, `Phase::Transform`. Це не scene graph Node*. Для заявленого 3D/editor picking — правильна форма.

Рендер-команда. Варіант з 2 вирос до 4: `CmdDrawMesh`, `CmdDrawUI`, `CmdDrawParticles`, `CmdSetView`. `CmdSetView` несе камеру й світло — потрібен для viewport редактора (`render_world_viewport`) і кількох вікон.

UI backend. NanoVG з публічного/приватного шляху гри зник. Є `src/ui/render/` (власний OpenGL device). Ігри малюють кастом через `IDrawList` / `IPaint`, не через `IUiPainter`.

Що зберегти

- Стіна `include/engine/` vs `src/`. Гра не бачить glad/SDL/spdlog/tinyxml2.
- Закритий command buffer без `CmdCustomDraw`.

- GUID `AssetsDb`, sidecar `.meta`, codegen, що падає без meta.
- Headless `Host` + ті самі `flush_worlds` / `simulate_worlds`.
- Bevy-подібні `Events<T>`, named `ActionId`.
- XML + CSS + MVVM, без `onClick`.
- `OverlayMode` на `IWindowControl` (Auto / AlwaysEnabled / AlwaysDisabled).
- Play/Stop контракт модуля: ніщо з коду гри не переживає unload (задокументовано й покрито тестами editor).
- Нормативні сторінки Principles / Scope / Boundaries — рідкість; їх варто тримати синхронними з кодом.

Поточні проблеми (за важливістю)

Оцінка серйозності: High / Medium / Low. Це не баги CI, а місця, де архітектура роз'їдеться за 2–3 роки.

1. Medium–High — `World::ctx<T>()` лишився мішком композиції

`EngineServices` прибрав service locator рушія. Світ як і раніше тримає довільні ресурси: `SpriteMaterialCache`, купа UI-транзієнтів (`UiFocusState`, `UiActiveDrags`, `UiClipboard`, `UiInspector`, …), `Presentation*`, `IFatalError*`.

Це EnTT-shaped і свідомо. Проблема не в самому `ctx`, а в тому, що немає шару «світові сервіси vs транзієнтний UI-стан». Новий жест UI = новий тип у `ctx`. `SpriteMaterialCache` ключується сирим `ITexture*` і не евиктиться, поки живе світ.

Не ламати `ctx`. Винести процесні речі (вже зроблено для `Presentation`) і явно позначити caches як owned render state з політикою життя.

2. Medium — публічне API несе форму редактора

`include/engine/engine.h` тягне `ui/inspector.h` і `ui/profiler.h`. `CmdDrawUI` містить inspector hover/selection, profiler canvas entity, popup layer. Dock float відкриває OS-вікна з `EngineSystemDeps::windows`.

Principles §13: інструменти живуть у редакторі, рушій тримає дані, які вони читають. Це майже так, але дані розмазані по command buffer і umbrella-заголовку гри. Експортована гра компілює inspector-поля завжди, «щоб layout команди не змінювався».

Наслідки: кожен новий інструмент редактора спокушає додати поле в `CmdDrawUI` або тип у `include/engine/ui/`. Через рік public surface = editor protocol.

Краще: окремий overlay/debug channel або умовна секція команди, якої Shipping player не бачить навіть як поле.

3. Medium — рендер-абстракція все ще не шов бекенда

Гра не кличе OpenGL — це виконано. Бекенд усе ще робить `dynamic_pointer_cast<OpenGLShader/Mesh/Texture>`. DirectX/Metal у Scope як і раніше out. Поки один бекенд — це борг, не блокер.

`CmdSetView` + матеріали + sort — правильний напрям. Шов зламається в день другого GPU API, не раніше. Тримати `IMesh`/`IShader` без RTTI-cast варто як правило «не регресувати», не як поточний епік.

4. Medium — `camera.h` залежить від UI

`include/engine/ecs/camera.h` інклудить `ui/canvas.h` лише заради `WindowSize`. Aspect/projection камери не є UI-поняттям. `WindowSize` сидить у UI-заголовку з історичних причин.

Це той самий шарний інверт, що був у першому аудиті. Зараз він менший (немає dual ctx), але include-граф ECS→UI лишився. Винести `WindowSize` (або `Extent2`) у `core` / `render`.

5. Low–Medium — overlay усе ще first-class у windowed runtime

Політика окрема, `OverlayMode` публічний, default Auto = «є прозоре вікно ⇒ хуки». Для desktop overlay-ігор це фіча. Для звичайного вікна — мертвий код на гілці Auto.

Не критично. Не робити Auto дефолтом для не-прозорих ігор уже вміють (`AlwaysDisabled`). Документувати в гайді гри, не в ядрі циклу.

6. Low–Medium — закритий `Phase` (тепер 8)

Додали `Transform`. Це правильно. Далі кожна нова «коли це бігти» спокуса — нове значення enum. Розкол `register_*` зменшив шкоду: не всі світи реєструють усі фази. Сам enum лишається закритим.

Для 2D-рушія 8 фаз — нормально. Плагінних фаз не треба, поки ігри вкладаються в Game/Fixed.

7. Low — ownership GPU й сирі UI-вказівники

`Renderable` / `Sprite` / `CmdDrawMesh` тримають `shared_ptr` на GPU-об'єкти. `CmdDrawUI` тримає сирі `UiDocument*` / `Stylesheet*`. Це той самий контракт «буфер живе один кадр». Працює, поки ніхто не зберігає команду за кадр.

`SpriteMaterialCache` — ще один shared_ptr-кеш без unload окремо від світу.

8. Low — модель помилок задокументована, але широка

Boundaries таблиця fatal vs `expected` vs warn — добре. На практиці: HTTP init лише лог+`Unsupported`; missing extra stylesheet skip; Sprite без mesh/material — fatal. Гра має читати таблицю, не здогадуватись. Це вже не хаос першого аудиту.

Три речі на горизонт 2–3 років (оновлено)

1. Overlay як default runtime — знято з цього списку. Є політика й публічний opt-out.

2. Безсмертні ресурси — сильно пом'якшено `unload_catalog` + lazy UI. Залишок: `SpriteMaterialCache` і GPU `shared_ptr` на компонентах. Для editor Play/Stop цього достатньо; для довгих сесій з стрімінгом ассетів — ні.

3. Закритий schedule + `ctx` — schedule покращився (розкол реєстрації, `Worlds`). `ctx` як мішок UI-стану погіршився кількісно (більше жестів = більше типів). Новий №1 на горизонті: редакторні поля в публічному command/API. Якщо кожен інструмент (gizmo, 3D viewport overlay, asset drop) додає поле в `CmdDrawUI` або тип в `engine.h`, exported game і editor назавжди зшиті layout'ом команди.

Engine vs game vs editor

```
SDK / wind_editor          engine.dll (ENGINE_EDITOR)
        |                        |
        | Play: load copy        | EngineHost, Worlds, services
        v                        v
   game.dll  ---- C ABI ----  wind_create_game(EngineServices)
        |                        |
        | Stop: destroy worlds, clear loaders, unload_catalog, FreeLibrary

Експорт: static libengine + game sources, без модуля.
```

Це правильна межа. Тести editor (`play_session_test`, `edit_module_test`) фіксують порядок unload — це архітектурний тест, не косметика.

Тертя:

- `ENGINE_EDITOR` є `PUBLIC` define. Principles кажуть: гра тестує його лише для tools. Це дисципліна, не компілятор. Регресія «if EDITOR then інший HUD» не спіймається рушієм.
- Inspector/profiler символи доступні Shipping-гравцю як типи (profiler call sites у Shipping — no-op через `ENGINE_UI_PROFILER`). CmdDrawUI layout не стискається.

HTTP, process launcher, capture, file dialog, CLI server — in scope як сервіси хоста. Гра їх бачить через `EngineServices`. Це не просочування однієї гри; це platform layer. Capture (фізична камера, wind-282)

виглядає нішевим, але шов `ICapture` + updatable texture — нормальний engine service, не геймплей.

Відповідність старим знахідкам

Стара знахідка	**Статус на** 14b1945
Overlay зварений у головний цикл	Політика + public `OverlayMode`. Low–Medium
Dual `WindowSize` / `MouseConsumed.value`	Закрито. Процесна `Presentation`
Eager preload усього UI-каталогу	Закрито. Lazy + builtin font
Immortal `AssetsDb`	Частково: є `unload_catalog`
Обов’язковий монолітний `register_engine_systems`	Частково: є split + `enable_ui/audio`
`EngineRuntime` god + дубль Host	Частково: `EngineHost` спільний; runtime лишився вікном/GL/loop
Мертвий `IGame::on_draw`	Закрито
`kApiEpoch` не росте	Замінено `build_id`
Немає editor / networking / hierarchy	Тепер in scope, і зроблені
Рендер не шов бекенда	Без змін по суті (`dynamic_pointer_cast`)
`camera.h` → UI	Без змін по суті
GPU `shared_ptr` / сирі UI pointers	Без змін по суті
`SpriteMaterialCache` у `ctx`	Живе

Висновок

Питання першого аудиту було: *це узагальнений рушій чи випадкова архітектура поточної гри?*

На `14b1945` відповідь: узагальнений 2D-first рушій, свідомо зібраний навколо editor-host і SDK. Це не обвинувачення. Principles §8 і §13 саме це й вимагають.

Не викидати спрайти, аніматор, партикли, 9-slice, math-in-UI, dock, HTTP. Викидати не треба нічого з цього списку як «зайве». Стежити треба за трьома швами:

1. Реєстрація систем лишається опційною (не звалити все назад в один обов’язковий бандл без `enable_*`).

2. Інструменти редактора не роздувають публічний `Command` / `engine.h`.

3. `ctx` не стає єдиним місцем для будь-якого нового стану.

Якщо ці три шви втримати, Wind масштабується як production engine. Якщо ні — через два роки це буде «редактор, з якого важко зібрати чистий player», а не «гра в ядрі».

Джерела: `docs/tech/architecture/{Principles,Scope,Boundaries,Overview,Runtime Loop}.md`, `include/engine/{engine.h,igame.h,ecs/systems.h,core/engine_host.h,core/engine_services.h,core/worlds.h,core/game_module.h,render/commands.h,ui/presentation.h}`, `src/ecs/systems.cpp`, `README.md`.